

Trouve ton artisan

Plateforme des artisans Auvergne-Rhône-Alpes

Devoir bilan

Centre Européen de Formation

Développement Web – Projet React.js

Document réalisé dans le cadre du devoir bilan – CEF

Sommaire

1. Contexte du projet

2. Maquettes Figma

3. Base de données (MCD / MLD)

4. Mesures de sécurité

5. Veille sécurité

6. Liens du projet

1. Contexte du projet

1.1 Présentation de l'entreprise

La Région Auvergne-Rhône-Alpes couvre 12 départements dans le quart sud-est de la France. Elle dispose de bureaux à Lyon et Clermont-Ferrand. Avec près de 221 000 entreprises d'artisanat en 2021, c'est l'une des régions les plus artisanales de France. La Région souhaite valoriser ce secteur et faciliter la mise en relation entre particuliers et artisans.

1.2 Expression des besoins

La Région a souhaité la création d'une plateforme web permettant aux particuliers de :

- Trouver un artisan selon sa catégorie (Bâtiment, Services, Fabrication, Alimentation) ;
- Consulter les fiches détaillées des artisans ;
- Contacter un artisan via un formulaire (demandes de renseignements, prestations, tarifs).

Un jeu d'essais (fichier Excel) a été fourni pour alimenter la base de données. Chaque artisan appartient à une seule spécialité, elle-même rattachée à une seule catégorie.

1.3 Contraintes

- **Accessibilité** : Site accessible à tous (jeunes, seniors, personnes en situation de handicap), conformément à la norme WCAG 2.1.
- **Responsive** : Conception mobile first, adaptée à tous les écrans (mobile, tablette, ordinateur).
- **Sécurité** : Mise en place des bonnes pratiques pour une collectivité (validation, protection des données, CORS).
- **Identité visuelle** : Cohérence avec l'environnement numérique de la Région (police Graphik, palette de couleurs officielle).

1.4 Livrables attendus

- Maquettes Figma (mobile, tablette, desktop) ;
- Site frontend React utilisant une API ;
- API Node.js/Express interrogeant une base MySQL ;
- Script de création et d'alimentation de la base de données ;
- Site hébergé et accessible en ligne.

1.5 Vérification W3C

J'ai vérifié la conformité du code avec les validateurs du W3C. Les principales pages du site (accueil, liste des artisans par catégorie, fiche détaillée, pages légales et page 404) ont été passées au [validateur HTML](#), et les feuilles de style au [validateur CSS](#). Les contrôles ont été effectués sans erreurs ni avertissements. Les seuls messages affichés sont des informations liées à la syntaxe générée par React (balises vides avec slash), ce qui est attendu pour une application React et n'affecte pas la conformité du document.

Document réalisé dans le cadre du devoir bilan – CEF

2. Maquettes Figma

Les maquettes ont été réalisées sous Figma pour les trois formats demandés : mobile, tablette et ordinateur. Elles reprennent l'identité graphique de la Région (police Graphik, logo, palette de couleurs) et définissent la structure de toutes les pages : accueil, liste des artisans par catégorie, fiche détaillée, formulaire de contact, pages légales et page 404.

Lien vers les maquettes

Maquettes Figma complètes :

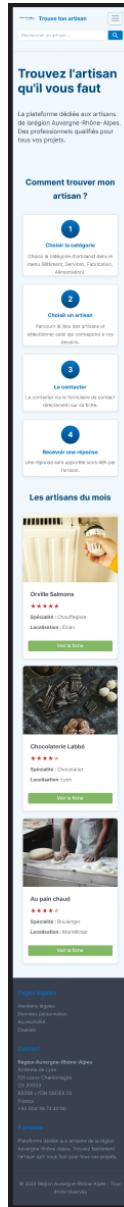
[Voir l'ensemble des maquettes sur Figma](#)

Captures d'écran

Les maquettes ont été réalisées pour l'ensemble des pages du site, en mobile, tablette et desktop. Voici trois captures représentatives :

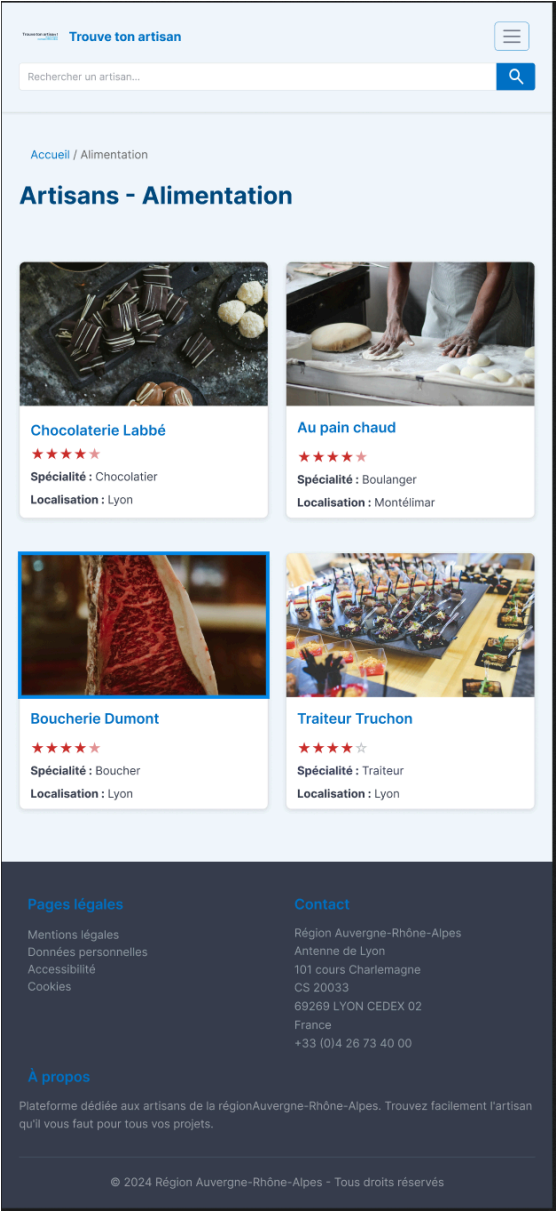
1. Maquette mobile – Page d'accueil

Trouve ton artisan – Devoir bilan | Centre Européen de Formation

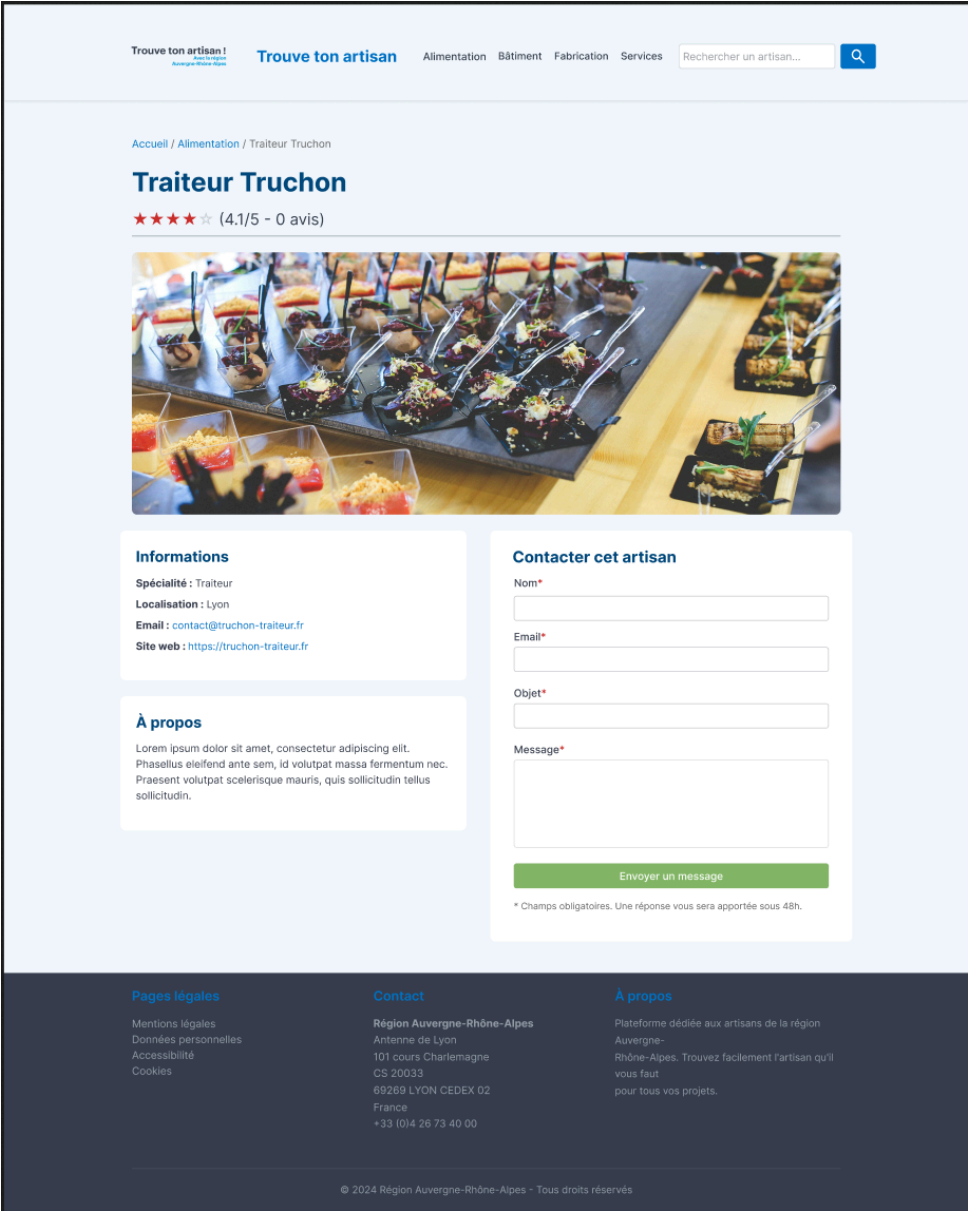


2. Maquette tablette – Liste des artisans

Trouve ton artisan – Devoir bilan | Centre Européen de Formation



3. Maquette desktop – Fiche artisan + formulaire de contact



3. Base de données (MCD et MLD)

3.1 Modèle Conceptuel de Données (MCD)

CATEGORIE (1,n) -----< SPECIALITE (1,n) -----< ARTISAN (1,n) -----< CONTACT

Entités :

- **CATEGORIE** : Catégorie d'artisanat (Bâtiment, Services, Fabrication, Alimentation)
- **SPECIALITE** : Spécialité au sein d'une catégorie (ex. Boulangerie, Plomberie)
- **ARTISAN** : Artisan avec ses informations (nom, email, note, localisation, etc.)
- **CONTACT** : Message envoyé via le formulaire de contact à un artisan

Relations :

- Une catégorie contient plusieurs spécialités (1,n)
- Une spécialité appartient à une seule catégorie (1,1)
- Une spécialité contient plusieurs artisans (1,n)
- Un artisan appartient à une seule spécialité (1,1)
- Un artisan reçoit plusieurs contacts (1,n)
- Un contact est envoyé à un seul artisan (1,1)

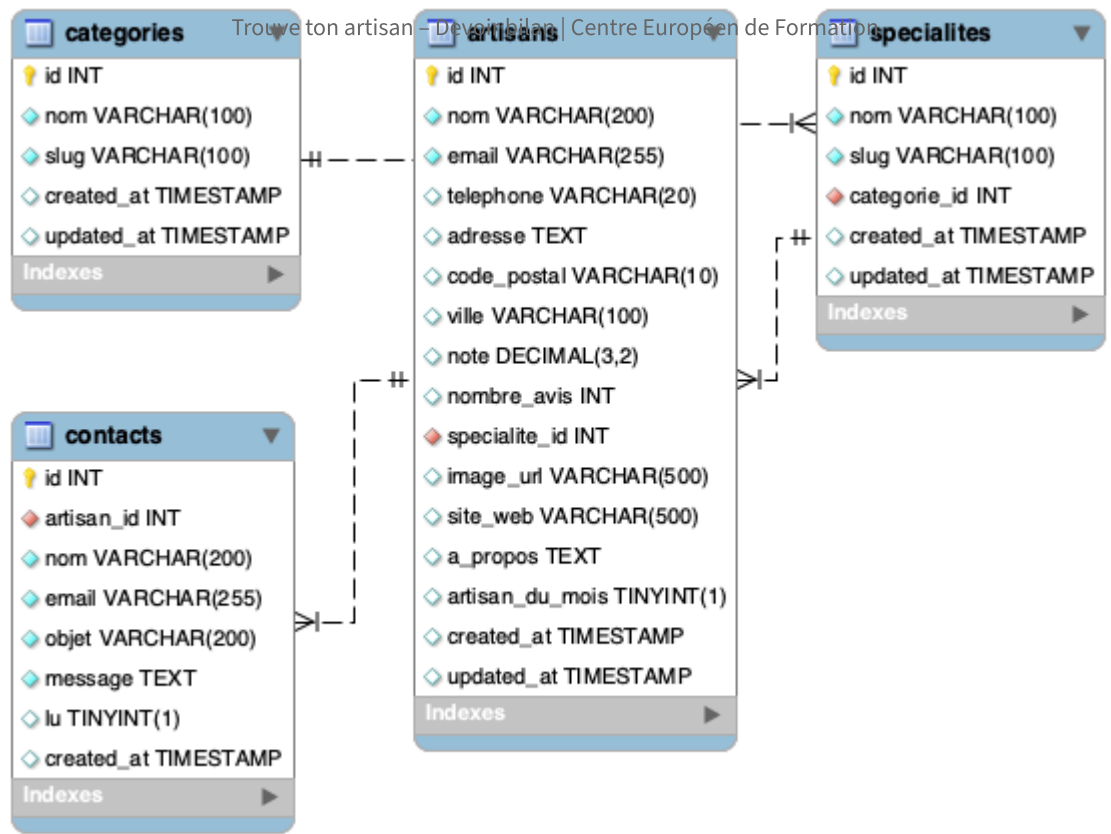


Figure 1 – Schéma MCD/MLD de la base de données « Trouve ton artisan »

3.2 Modèle Logique de Données (MLD)

Table categories

Colonne	Type	Contraintes
id	INT	PK, AUTO_INCREMENT
nom	VARCHAR(100)	NOT NULL, UNIQUE
slug	VARCHAR(100)	NOT NULL, UNIQUE
created_at, updated_at	TIMESTAMP	-

Table specialites

Colonne	Type	Contraintes
id	INT	PK, AUTO_INCREMENT
nom	VARCHAR(100)	NOT NULL
slug	VARCHAR(100)	NOT NULL
categorie_id	INT	FK → categories.id, ON DELETE CASCADE
UNIQUE (nom, categorie_id)		

Document réalisé dans le cadre du devoir bilan – CEF

Table artisans

Trouve ton artisan – Devoir bilan | Centre Européen de Formation

Colonne	Type	Contraintes
id	INT	PK, AUTO_INCREMENT
nom	VARCHAR(200)	NOT NULL
email	VARCHAR(255)	NOT NULL
telephone, adresse, code_postal, ville	VARCHAR/TEXT	NULL
note	DECIMAL(3,2)	CHECK (0–5)
specialite_id	INT	FK → specialites.id, ON DELETE CASCADE
image_url, site_web, a_propos	VARCHAR/TEXT	NULL
artisan_du_mois	BOOLEAN	DEFAULT FALSE
created_at, updated_at	TIMESTAMP	-
Index : idx_nom, idx_specialite, idx_artisan_du_mois		

Table contacts

Colonne	Type	Contraintes
id	INT	PK, AUTO_INCREMENT
artisan_id	INT	FK → artisans.id, ON DELETE CASCADE
nom, email, objet	VARCHAR	NOT NULL
message	TEXT	NOT NULL
lu	BOOLEAN	DEFAULT FALSE
created_at	TIMESTAMP	-

4. Mesures de sécurité mises en place

4.1 Protection CORS (Cross-Origin Resource Sharing)

Mise en œuvre : Le middleware `cors` est configuré pour n'autoriser que les requêtes provenant du domaine du frontend (en production : URL Vercel ; en développement : localhost:3000). Seules les méthodes GET, POST et OPTIONS sont autorisées.

Intérêt : Empêche les sites tiers d'interroger l'API et limite les attaques par requêtes cross-origin non autorisées.

4.2 Helmet

Mise en œuvre : Le middleware `helmet` sécurise les en-têtes HTTP (X-Content-Type-Options, X-Frame-Options, etc.) pour réduire les risques XSS, clickjacking et autres.

Intérêt : Renforce la sécurité par défaut des réponses HTTP du serveur.

4.3 Validation des données côté serveur (express-validator)

Mise en œuvre : Le formulaire de contact utilise `express-validator` pour valider nom (2–200 caractères), email (format valide), objet (3–200 caractères), message (10–5000 caractères) et `artisan_id` (entier requis).

Intérêt : Garantit l'intégrité des données et empêche l'injection de contenu malveillant ou des valeurs hors limites.

4.4 Protection contre les injections SQL

Mise en œuvre : L'accès à la base utilise Sequelize (ORM) avec des requêtes paramétrées. Pour la recherche par nom, les apostrophes sont échappées avant l'utilisation dans une requête brute.

Intérêt : Évite les injections SQL en ne concaténant jamais directement des entrées utilisateur dans les requêtes.

4.5 Gestion des erreurs en production

Mise en œuvre : En production, les messages d'erreur détaillés ne sont pas renvoyés au client ; seul un message générique est affiché pour ne pas exposer la structure du système.

Intérêt : Limite la divulgation d'informations sensibles en cas d'erreur.

4.6 Variables d'environnement

Mise en œuvre : Les mots de passe, clés et URLs de connexion sont stockés dans un fichier `.env` non versionné. En production, les variables sont configurées sur la plateforme d'hébergement.

Intérêt : Les identifiants ne figurent pas dans le code source et ne sont pas exposés sur Git.

5. Veille sécurité et vulnérabilités

5.1 Vulnérabilités étudiées

Injection SQL

Les ORM comme Sequelize utilisent des requêtes paramétrées, ce qui évite l'injection SQL. J'ai vérifié que les seules requêtes brutes (recherche par nom) échappent correctement les apostrophes et utilisent des placeholders.

XSS (Cross-Site Scripting)

React échappe par défaut le contenu affiché. Les données provenant de la base (noms, descriptions) sont affichées sans `dangerouslySetInnerHTML`, ce qui limite les risques XSS côté client.

CSRF (Cross-Site Request Forgery)

L'API est consommée par un frontend SPA. La restriction CORS limite les requêtes à notre domaine. Pour renforcer la protection, on pourrait ajouter des tokens CSRF sur les formulaires sensibles.

Données sensibles

Les mots de passe ne sont pas stockés dans cette application (pas d'authentification utilisateur). Les emails des artisans sont des données de contact professionnelles. Le formulaire de contact enregistre les messages en base ; l'envoi par email est optionnel et configurable.

5.2 Corrections et bonnes pratiques appliquées

- Limitation de la taille du corps des requêtes JSON (10 Mo) pour éviter les attaques par déni de service.
- Validation stricte des types (artisan_id en entier, emails au format valide).
- Pas d'exposition des chemins de fichiers ou de la structure interne du serveur en production.

6. Liens du projet

Repository GitHub :

<https://github.com/Laupaka2/devoir-bilan-trouve-ton-artisan>

Le dépôt contient : le code source (frontend React, backend Node.js), le script de création de la base (`01_create_database.sql`), les scripts d'alimentation (`02_insert_data.sql`, `05_noms_et_images_personnalisés.sql`), ainsi qu'un script d'import depuis Excel (`import-excel.js`) pour les données fournies.

Site en ligne :

<https://devoir-bilan-trouve-ton-artisan.vercel.app/>

Document rédigé dans le cadre du devoir bilan – Centre Européen de Formation.